@SpringBootApplication & Startup

Q1. What does the @SpringBootApplication annotation include, and what happens inside the app when it starts?

A: The **@SpringBootApplication** annotation is a shortcut that combines three key annotations: **@Configuration**, **@EnableAutoConfiguration**, and **@ComponentScan**. When the app starts, the **IoC Container** performs auto-configuration, loads all bean definitions, and finally starts the embedded server (like Tomcat).

Q2. How does Spring Boot decide which beans to auto-configure?

A: Spring Boot uses **conditional annotations** to decide. It looks at the **classpath** to see what libraries are available. If a library like JPA is present, its corresponding auto-configuration class runs, checks its internal conditions (e.g., if a database connection is needed), and automatically creates the necessary beans.

Q3. Can you change or override parts of auto-configuration? How do you do it?

A: Yes, you can override auto-configuration by simply creating your own custom bean with the **same type or name** as the one Spring created. Spring always prefers your explicitly defined bean over the auto-configured default. You can also tweak auto-config behavior by setting specific properties in your configuration files.

Q4. How do you turn off a specific auto-configuration class?

A: You can disable any auto-configuration class by using the **exclude** attribute within the **@SpringBootApplication** annotation, specifying the class you want to turn off. Alternatively, you can list the class name(s) under the spring.autoconfigure.exclude property in your application configuration file.

Q5. What is the difference between @SpringBootApplication and @EnableAutoConfiguration?

A: @EnableAutoConfiguration only turns on the auto-configuration process. **@SpringBootApplication** is the main launch annotation that bundles three functions: it enables auto-configuration, enables component scanning, and marks the class itself as a source of bean configuration.

Runtime Visibility & Configuration Loading

Q6. How can you list or print all beans that Spring Boot has created?

A: You can list all created beans by injecting the **ApplicationContext** into a component and calling its getBeanDefinitionNames() method. The Actuator /beans endpoint also provides a detailed, live list of all loaded beans, their scope, and their dependencies.

Q7. How can you check which environment properties are active in the running app?

A: You can check active properties by using the **Actuator /env** endpoint, which provides a full list of all loaded property sources and their active values. In code, you can inject the **Environment** object and call getProperty() for specific keys.

Q8. What is the order in which Spring Boot loads configuration values?

A: Spring Boot loads configuration values in a fixed priority order: **command-line arguments** are loaded first, then **environment variables**, followed by **application properties/yaml files**, and finally default values. Higher-priority sources always override lower-priority ones.

Q9. How can you find out which profiles are active while the app is running?

A: You can find active profiles by injecting the **Environment** object and calling environment.getActiveProfiles(). The **Actuator /env** endpoint also lists the currently active profiles explicitly.

Application Startup & Runners

Q10. What is CommandLineRunner, and when do you use it?

A: CommandLineRunner is an interface used to execute a specific piece of code right after the Spring application context has successfully started. You use it primarily for running one-off tasks during startup, such as logging initial configuration details, loading test data into the database, or performing initial health checks.

Q11. What is ApplicationRunner, and how is it different from CommandLineRunner?

A: ApplicationRunner also runs code at startup, making it functionally similar to CommandLineRunner. The main difference is that ApplicationRunner gives you access to the command-line arguments in a more structured way (as an ApplicationArguments object) instead of a simple string array.

Q12. What is the Spring Boot CLI, and what common commands does it support?

A: The **Spring Boot CLI (Command Line Interface)** is a tool that lets you quickly write and run small Spring Boot applications using **Groovy scripts** without setting up a full Java project. Common commands include **spring run** (to execute a script), **spring init** (to create a new project), and **spring install** (to add dependencies).

Q13. Can you run a Spring Boot app without a web server? How?

A: Yes, you can run a Spring Boot application without a web server. You do this by setting the application type to **WebApplicationType.NONE** in the main class before running it, or by excluding the web starter dependency from your build file.

Q14. How do you create a Spring Boot app that is not a web application, like a batch job?

A: You create a non-web application by setting the application type to **WebApplicationType.NONE** in the SpringApplicationBuilder. You would then typically use a **CommandLineRunner** or **ApplicationRunner** to execute the main business logic (like a batch process) and then gracefully exit the application.

Profiles and Configuration

Q.15. How do profiles work in Spring Boot?

A: Profiles (@Profile("dev")) act as conditional tags for beans and configuration settings. They allow you to define different versions of your code or configuration for specific environments (like dev vs. prod). Only the beans and configuration marked with the **active profile** are loaded when the application starts.

Q.16. Why do real projects use profiles?

A: Real projects use profiles to keep environments **secure and stable**. Profiles ensure that sensitive production secrets (like API keys or specific database URLs) are never accidentally loaded or used when a developer is running the application locally. They make it easy to switch settings without changing the core code.

Q.17. Can application.properties and application.yml be used together? How does Spring pick the values?

A: Yes, they can be used together. Spring loads all configuration files from the current active profile. If a property is defined in both files, the value loaded later or from a source with higher priority (like command line) will **override** the others.

Q.18. What is good about YAML, and what problems can it cause?

A: YAML is good because it uses indentation to structure configuration data, making nested properties much cleaner and easier to read than the flat dot notation of .properties files. However, its main problem is that it is **indentation sensitive**; incorrect spacing can lead to parsing errors that are difficult to debug.

Q.19. What steps do you follow to safely move from properties files to YAML?

A: Follow these steps: 1) Convert configuration one file at a time, 2) Use online converters to help with initial formatting, and 3) Test the conversion thoroughly, especially paying attention to **lists and complex nested structures**, as YAML formatting is very sensitive to precise spacing.

Q.20. How does Spring Boot handle different configs for dev, QA, stage, and prod?

A: Spring handles different configs using **profile-specific files**, such as **application-dev.yml** and **application-prod.yml**. The core application loads defaults from application.yml, and then the specific profile file overrides only the necessary properties for that environment.

Security & Request Flow

Q.21. How does an HTTPS request move through a Spring Boot application?

A: The HTTPS request first hits the **embedded server** (like Tomcat), where the SSL handshake handles the initial decryption of the encrypted data. The decrypted request then passes through Spring Security filters for authentication and authorization. Finally, the **DispatcherServlet** routes the request to the correct Controller method for processing.

Q.22. Can you turn off the embedded server if your app doesn't need networking?

A: Yes, you can turn off the embedded server. You do this by setting the application type to **WebApplicationType.NONE** in the main class before running it. This creates a non-web application (like a batch job) that only runs Spring's core logic.

Q.23. What is the difference between deploying a WAR file vs using the built-in server?

A: Deploying a **WAR file** requires you to install and manage an external application server (like a separate instance of Tomcat or JBoss) where your application lives. Using the **built-in server** (JAR file) is simpler because the server is bundled inside your application, making the application fully self-contained and easier to deploy in cloud environments.

Data Access & Transactions

Q.24. How does Spring Boot make working with databases easier using Spring Data JPA?

A: Spring Data JPA makes database access easier by eliminating boilerplate code for basic queries. You simply define an **interface** that extends JpaRepository, and Spring automatically generates common CRUD (Create, Read, Update, Delete) methods and simple custom queries for you based on the method name.

Q.25. How do you handle situations where multiple beans have the same type?

A: You handle this ambiguity by using the **@Qualifier** annotation to explicitly specify the name of the bean you want to inject (e.g., @Qualifier("mainDatabase")). Alternatively, you can use the **@Primary** annotation on one bean to mark it as the default choice.

Q.26. What are good practices when using @Transactional?

A: Good practice is to apply **@Transactional** to the **Service layer** methods, not the Controller or Repository. Keep transactional methods as short as possible to minimize the time database locks are held. Only adjust advanced settings (like isolation levels) when necessary for specific concurrency requirements.

Exception Handling

Q.27. How do you create a common place for handling exceptions using @ControllerAdvice and @ExceptionHandler?

A: You create a global handler by making a dedicated class annotated with **@ControllerAdvice**. Inside this class, you define specific methods annotated with **@ExceptionHandler** that map to certain exception types (like ResourceNotFoundException). This centralizes your error logic, ensuring all controllers return a consistent, standard error response to the client.

Q.28. How does Spring Boot automatically map errors using ErrorController?

A: Spring Boot automatically registers a fallback **BasicErrorController** that handles all errors not caught by a specific @ExceptionHandler. This controller maps all uncaught errors to the /error path and returns a default JSON or HTML error view. This prevents clients from seeing raw server stack traces when an error occurs.

Validation

Q.29. How does Spring Boot support validation using Hibernate Validator?

A: Spring Boot automatically includes and configures Hibernate Validator, which provides the reference implementation for the Bean Validation specification. You apply validation rules by adding standard annotations (like @NotNull or @Size) to the fields of your request object (DTO). The validation is triggered by adding the @Valid annotation to the method parameter in your controller.

Q.30. Can you build your own custom validator?

A: Yes, you can build a custom validator. You do this by defining a new annotation (which represents your validation rule) and creating an implementation class that implements the ConstraintValidator interface. This implementation class contains the specific Java logic that checks if the field's value meets your custom business rule.

Q.31. How do you validate two or more related fields together?

A: To validate related fields (like ensuring a start date is before an end date), you apply the custom validator annotation at the class level instead of the field level. The validator implementation then gets access to the entire object and can compare both fields together within its single check.

Q.32. How do you handle form validation in Spring Boot?

A: You handle form validation by putting validation annotations (like @NotEmpty) on your request object (DTO). You add the @Valid annotation to that object in the controller method signature. Validation errors are automatically collected in a special BindingResult object, which you can then check for errors.

Q.33. Can you use outside validation libraries like Apache Commons Validator?

A: Yes, you can use outside libraries by manually integrating them into your project. However, it is usually simpler to leverage Spring's built-in Validator interface and use the standard Hibernate Validator annotations, as they are seamlessly integrated into the framework.

RESPONSE HANDLING

Q.34. What is the difference between returning ResponseEntity and returning a normal object?

A: Returning a normal Java object relies entirely on default settings and only returns the data (JSON/XML) with a standard 200 OK status code. Returning **ResponseEntity** gives you full control to manually customize the **HTTP status code** (e.g., 201 Created, 404 Not Found), headers, and the exact response body content, which is crucial for building robust REST APIs.

LOGGING & MONITORING

Q.35. How do you set up logging in Spring Boot?

A: Logging is set up automatically using **Logback** by default, requiring no initial configuration. You configure logging behavior by defining properties prefixed with **logging.level** (e.g., logging.level.root=INFO) in your application.properties or by providing a custom logback-spring.xml file.

Q.36. What is SLF4J, and why does Spring prefer using it?

A: SLF4J (Simple Logging Facade for Java) is not a logging framework; it is an **abstraction layer** or façade. Spring prefers using it because it allows developers to write logging calls without committing to a specific implementation. This lets you switch between underlying logging systems (like Logback or Log4j2) easily without changing your application code.

Q.37. If you want to switch from Logback to Log4j2, what changes do you need?

A: To switch from Logback to Log4j2, you must modify your build file (pom.xml or build.gradle). You need to **exclude** the default Logback dependency that comes with the web starter and explicitly **include** the necessary Log4j2 starter and bridging dependencies.

Q.38. What is Spring Boot Actuator, and how do you turn it on?

A: Actuator is a module that adds production-ready monitoring and management features to your application (like seeing internal metrics or running health checks). You turn it on by including the **spring-boot-starter-actuator** dependency in your build file.

Q.39. Which Actuator endpoints are most useful for debugging?

A: The most useful endpoints for debugging are **/health** (shows component status), **/info** (shows custom application details), **/env** (shows all active properties), and **/beans** (lists all loaded beans in the container). You often need to enable them explicitly in configuration.

DevTools & Developer Experience

Q.29. Why do developers use Spring Boot DevTools?

A: Developers use DevTools primarily for the **automatic restart** feature. It detects code changes in the project and quickly restarts the application context. This saves the developer significant time and effort of manually stopping and starting the server every time they modify code, greatly improving workflow speed.

Q.30. Why should DevTools never be enabled in production?

A: DevTools should **never** be enabled in production because they introduce serious **security risks** and performance overhead. Features like the automatic restarting functionality and

enabling the H2 console are strictly designed for a local development environment and can be exploited if deployed live.

Q.31. How do DevTools help speed up development when code changes often?

A: DevTools speed up development by using a **secondary classloader** to load the changed classes. This means only the modified code needs to be reloaded, resulting in a much faster restart cycle than a full, cold application restart. It effectively implements a fast "live reload" functionality.

Testing

Q.32. How do you test a Spring Boot application (unit tests, slice tests, integration tests)?

A: You test by creating a testing pyramid: **Unit Tests** verify single classes using Mockito. **Slice Tests** verify specific layers (e.g., Controller or Repository). **Integration Tests** verify the entire component stack (using @SpringBootTest) to ensure all layers communicate correctly.

Q.33. What does @SpringBootTest do?

A: @SpringBootTest is used to load the full Spring ApplicationContext. It is essential for integration tests because it creates and configures all beans and components, allowing you to test the complete application stack as if it were running live.

Q.34. What is @MockBean used for in Spring Boot tests?

A: @MockBean is used specifically to create a Mockito mock object and inject it into the application context, **replacing a real bean**. This allows you to isolate the component under test and control the behavior and results of its external dependencies (like a third-party service).

Q.35. How do you mock external REST calls (like using MockRestServiceServer or WireMock)?

A: You mock external REST calls by using **MockRestServiceServer** (for RestTemplate) to define expected requests and mock responses within the test context. Alternatively, **WireMock** can be used to set up a completely fake HTTP server that simulates the external service's behavior more realistically.

Q.36. How do you mock another microservice in a Spring Cloud setup?

A: The most effective way is to use **WireMock** to set up a fake server that simulates the external microservice's endpoints. This allows you to test your service's error handling and response logic in isolation without requiring the actual external service to be running.

Q.37. How do you write unit tests for Spring MVC controllers?

A: You write unit tests using **@WebMvcTest** and **MockMvc**. @WebMvcTest loads only the web layer. You use MockMvc to perform mock HTTP requests (like .perform(get("/api/user"))) and assert the expected HTTP status code and response content.

Q.38. What is the difference between JUnit 4 and JUnit 5, and why is JUnit 5 preferred today?

A: JUnit 5 is preferred today because it is modular, supports modern Java features, and allows more powerful testing structures. JUnit 4 used older, monolithic annotations (@Before, @After); JUnit 5 uses cleaner, more specialized annotations like @BeforeEach and @BeforeAll.

Deployment & Packaging

Q.39. What are the different ways to deploy a Spring Boot app (jar, war, container, cloud)?

A: Spring Boot applications are typically deployed as executable **JAR** files (the default), packaged into **Docker containers** for simple portability, or deployed directly to **Cloud** platforms (like AWS or Azure). Deployment as a WAR file is also possible but less common.

Q.40. How does Spring Boot make deployment easier than old Spring applications?

A: Spring Boot makes deployment easier because the application is fully **self-contained**. The application, its dependencies, and the server (Tomcat/Jetty) are bundled into a single executable JAR file, removing the need for complex external server setup and configuration.

Q.41. Can you deploy a Spring Boot app as a WAR on an external server? How?

A: Yes, you can deploy a Spring Boot app as a WAR file by changing the packaging type in pom.xml to war and modifying the main class to extend **SpringBootServletInitializer**. This is only done if you are forced to deploy to an existing external application server environment.

Dependency Management & Configuration

Q.42. What does Spring Boot mean by dependency management (BOM and version alignment)?

A: Dependency management means Spring Boot provides a **Bill of Materials (BOM)** that defines a specific set of compatible versions for all its major dependencies. This ensures that

when you include different starters, their transitive dependencies align, preventing version conflicts.

Q.43. What is the purpose of spring-boot-starter-parent?

A: The starter parent's purpose is to inherit default configurations and, most importantly, provide the **version management** (the BOM). By inheriting this, you don't need to specify version numbers for standard Spring dependencies in your own pom.xml.

Q.44. How can you change Jackson settings for the whole application?

A: You change Jackson settings (the JSON library) for the whole application by defining properties prefixed with **spring.jackson** in your configuration file (e.g., spring.jackson.serialization-inclusion=non_null).

Q.45. How does Spring Boot load values into a class using @ConfigurationProperties?

A: @ConfigurationProperties is used to bind external configuration values (from .yml or .properties files) directly to fields within a custom Java POJO (Plain Old Java Object). This is much cleaner than using many individual **@Value** annotations for complex configuration blocks.

Q46. How can you bind nested configuration values to Java POJOs?

A: You bind nested configuration values by applying the **@ConfigurationProperties** annotation to a class and defining fields within that class that are themselves custom Java POJOs. Spring automatically binds the nested structure based on the property prefix and the field names.

Q.47. What does @ConditionalOnMissingBean do, and when should you use it?

A: This annotation tells Spring to create a default bean **only if** a bean of that type has not already been created by the user or another configuration class. You use it in auto-configuration to provide sensible defaults without overwriting user-defined components.

Q.48. How do you override default validation messages?

A: You override default validation messages by creating a **ValidationMessages.properties** file in your classpath. Inside this file, you define custom messages that correspond to the keys of the standard validation annotations (e.g., javax.validation.constraints.NotNull.message).

Q.49. What are Slice Tests like @WebMvcTest, @DataJpaTest, @JsonTest, and why do we use them?

A: Slice Tests load only a small "slice" of the application context, focusing on one layer. We use them because they are much **faster** than full integration tests. **@WebMvcTest** tests controllers, **@DataJpaTest** tests repositories, and **@JsonTest** tests JSON serialization/deserialization.